# Autonomous QA — Backend × Provider-Mix Matrix Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development to implement this plan. Build work parallelizes across subagents; the emulator/backend RUNTIME loop is physically sequential (one emulator, one backend at a time) and runs inline. Steps use checkbox (`- [ ]`) syntax.

**Goal:** A fully autonomous QA session that, one backend at a time (Go `lava-api-go`, then Android `:api-app`), runs every provider-mix iteration through a fresh Lava install → onboarding → `1080p/mp3` search → open details → obtain a real download resource (magnet/http/.torrent), recording video + live logs throughout, vision-analyzing the recordings for defects, fixing what breaks, and looping until all iterations produce real, bluff-free evidence — then distributing new dev+prod Android builds.

**Architecture:** Reuse the existing containerized-KVM `emulator-matrix` runner, `record-device-session.sh`, the C05/C06/C20 Challenges, and the HelixQA vision-OCR banks. Add (1) a parameterized onboarding+search+download Challenge keyed on `{backend, provider-subset, query}`, (2) a backend-swap orchestrator that brings up exactly one backend, runs the subset matrix with fresh installs, records, and tears down, (3) per-iteration evidence aggregation + vision analysis + a fix loop, (4) a §6.AK cycle-coverage map gating the distribute.

**Tech Stack:** Kotlin/Compose + JUnit4 + Orbit (client), Go (lava-api-go), podman 5.7.1 + /dev/kvm (containerized emulators), ffmpeg + adb screenrecord/logcat (recording), HelixQA YAML banks + vision OCR, bash orchestration (Local-Only CI/CD).

## Global Constraints

- **Anti-Bluff (§6.J/§6.L/§6.AK):** every iteration's PASS must rest on real user-visible state (rendered result row, `DownloadState.Completed(uri)`, the resolved magnet/http/.torrent) captured on a real emulator against a real

backend + real tracker. C00-only / cold-start-only evidence NEVER green-lights a distribute (§6.AK).
- **Credentials (§6.H):** the 5 tracker logins live ONLY in gitignored `.env` (already done). NEVER echo to logs, screenshots, commits, or `${VAR:-...}` shell expansions (§6.L 67th-cycle leak).
- **Recordings gitignored:** raw video/logcat/API-logs under `.lava-ci-evidence/autonomous-qa/**/raw/` + `device-recordings/` are git-excluded (done in `.gitignore`); only curated `summary.md`/`attestation.json`/`vision-analysis.md` are committed.
- **One backend at a time:** never run the Go API and the on-device `:api-app` backend concurrently for the same iteration.
- **Emulators in containers/VMs only (§6.AH/§6.AG):** gate runs use `emulator-matrix --runner=auto` (Linux+/dev/kvm → containerized+KVM). NEVER a host-direct emulator, NEVER a live ADB device.
- **No sudo/su (§6.U). No hosted CI. No hardcoded host:port/UUID/creds (§6.R). No host suspend/poweroff. Mirrors = GitHub + GitLab only (§6.W).**
- **Matrix (operator-chosen):** all 31 non-empty subsets of {RuTracker, RuTor, IPTorrents, NNMClub, Kinozal} × 2 backends × {`1080p`,`mp3`} = 124 functional iterations on `CZ_API34_Phone`. Full §6.AE multi-AVD matrix only at the pre-distribute gate.
- **IPTorrents:** Cloudflare-protected → Jackett+FlareSolverr sidecar for the Go API; on `:api-app` attempt-and-log-real-failures (operator-chosen; findings, not skips).
- **Distribute:** auto dev + prod once green AND §6.AK gate passes; operator pre-authorized the combined two-stage compression (record `combined-distribute-authorization` in §6.Z evidence).
- **§6.S:** update `docs/CONTINUATION.md` in the same commit as each phase transition.

---

# File Structure

### Create:

- `scripts/autonomous-qa/run-matrix.sh` — top orchestrator: backend up → subset loop (fresh install + record + run Challenge) → backend down → next backend.
- `scripts/autonomous-qa/lib-backend.sh` — bring Go API / `:api-app` up+down, health-probe, expose reachable URL for the onboarding cloud-input.
- `scripts/autonomous-qa/lib-subsets.sh` — emit the 31 non-empty provider subsets + per-subset evidence slug + state-hash.
- `scripts/autonomous-qa/vision-analyze.sh` — run HelixQA vision/OCR over each recording, emit `vision-analysis.md` findings.

- `scripts/autonomous-qa/aggregate-evidence.sh` — roll per-iteration attestations into a cycle `summary.md` + §6.AK `cycle-coverage-map-<ver>.yaml` + `<ver>-test-evidence.json`.
- `app/src/androidTest/kotlin/lava/app/challenges/Challenge70AutonomousQaProviderMatrixTest.kt` — parameterized onboarding→search→details→download Challenge reading `{backend, providers, query}` from instrumentation args.
- `lava-api-go/qa/banks/lava-matrix-<backend>-<subset-hash>.yaml` — generated per-iteration HelixQA vision banks (gitignored generated; template tracked).
- `.lava-ci-evidence/autonomous-qa/` — evidence root (raw gitignored, curated tracked).

**Modify:**

- `scripts/run-helixqa-provider-qa.sh` — accept a backend + subset selector (or call the new orchestrator).
- `app/build.gradle.kts` — ensure `RUTOR_*` + `NNMCLUB_*` BuildConfig fields exist (RuTracker/Kinozal/IPTorrents already wired).
- `docs/CONTINUATION.md` — §6.S phase tracking.
- `CHANGELOG.md` + `app/build.gradle.kts` + `api-app/build.gradle.kts` + `lava-api-go/internal/version/version.go` — §6.Y version bumps before distribute.

---

# Phase 0 — De-risk vertical slice (PROVE the keystone before scaling)

Goal: one fully real, recorded iteration — Go API up → fresh client install on a containerized KVM emulator → onboard **RuTracker only** (real creds) targeting the Go API via the onboarding cloud-input → search `1080p` → open a result → obtain a magnet/.torrent → video + logcat + API logs captured → vision-confirm. If any sub-step is blocked (container image missing, TLS/auth handshake fails, datacenter IP blocked by the tracker), STOP and surface it honestly — do not fabricate a pass.

- [ ] **0.1 Build lava-api-go + bring the Go backend up.** `cd lava-api-go && make build` (or `go build ./...`); `docker compose -f docker-compose.yml up -d`; poll `curl -ksf https://127.0.0.1:8443/health` until JSON liveness. Capture the host-reachable URL the emulator will use (`https://10.0.2.2:8443`). Record API logs to `.lava-ci-evidence/autonomous-qa/<date>/go/raw/api-server.log`.
- [ ] **0.2 Build the client debug APK.** `./gradlew :app:assembleDebug` (App ID `digital.vasic.lava.client.dev`). Confirm `RUTRACKER_*` BuildConfig populated from `.env` (gradle reads project props / `.env`).

**0.3 Boot ONE containerized emulator + smoke-prove the path.** Resolve the container image from `tools/lava-containers/vm-images.json`; run `scripts/run-challenge-matrix.sh --test-class lava.app.challenges.Challenge00CrashSurvivalTest --avds CZ_API34_Phone:34:phone --evidence-dir .lava-ci-evidence/autonomous-qa/<date>/phase0-smoke`. Expected: `real-device-verification.json` row `runner=containers-submodule`, `test_passed=true`. This proves container→install→instrument works on THIS host before any backend wiring.

- [ ]

**0.4 Prove backend reachability + TLS + auth from the emulator.** Add a minimal instrumented probe (or extend C44 ApiSearchAuth) that, on the booted emulator, hits `https://10.0.2.2:8443/health` and an authed route with the `Lava-Auth` key. Confirm: cert trust path works (the client must trust the Go API's self-signed cert — determine the mechanism: `usesCleartextTraffic` is true, but HTTPS needs a trust anchor; the cloud-input GoApi flow + `key` field carries the auth UUID). **This is the keystone — if it fails, the whole Go-backend half is blocked; record the exact failure and adapt (e.g. provision the cert into the emulator, or use the on-device api-app backend first).**

- [ ]

**0.5 Author the parameterized Challenge (RuTracker/1080p/GoApi instance).** Create `Challenge70AutonomousQaProviderMatrixTest.kt` reading instrumentation args `qa_backend`, `qa_providers` (CSV), `qa_query`; it drives onboarding (ApiSelection cloud-input = Go API URL → Providers select subset by displayName → Configure: form-login providers get `.env` creds via BuildConfig, anonymous-capable get the anon toggle → Summary "Start Exploring") → search `qa_query` → open result row → tap "Torrent"/"Magnet" → assert `DownloadState.Completed`/"Download completed" OR captured magnet URI. Include the §6.AB FALSIFIABILITY REHEARSAL KDoc block + `// covers-feature:` markers. Pass args via `-Pandroid.testInstrumentationRunnerArguments.qa_backend=goapi,qa_providers=rutracker,qa_query=1080p` (the emulator-matrix `--test-args` path).

- [ ]

**0.6 Run the single real iteration WITH recording.** Start `record-device-session.sh` for the emulator serial; run `Challenge70...` with args RuTracker/1080p/GoApi against the live Go backend + live rutracker.org (real creds). Stop recording. Outputs: `screen.mp4`, `logcat.txt`, `api-server.log` under the gitignored raw dir.

- [ ]

**0.7 Vision-confirm the recording.** Run the HelixQA vision/OCR step over `screen.mp4` (or sampled frames) asserting OCR goals: provider configured, results visible for `1080p`, topic/details opened, a download affordance + resolved link. Emit `vision-analysis.md`.

- [ ]

**0.8 Phase-0 verdict + commit.** If 0.1–0.7 all produced REAL evidence: write `.lava-ci-evidence/autonomous-qa/<date>/phase0-verdict.md` (curated, tracked) with the magnet/torrent obtained + video/log paths + vision findings; commit the harness + verdict (NOT raw media). If blocked: write the blocker honestly into the verdict + a `sixth-law-incidents` entry, surface to operator, do NOT proceed to Phase 1 with a fabricated pass.

**Phase 0 gate:** Do not start Phase 1 until 0.8 records a REAL pass (or the operator accepts a documented adaptation).

---

# Phase 1 — Generalize the iteration into the orchestrator

- [ ] **1.1** `lib-subsets.sh`: emit all 31 non-empty subsets of the 5 providers, each with a stable slug + state-hash.
- [ ] **1.2** `lib-backend.sh`: `backend_up goapi` (compose up + health + emit `https://10.0.2.2:8443`), `backend_up apiapp` (install+launch `:api-app` on the emulator, emit its loopback URL), `backend_down <which>`, mutual-exclusion guard (refuse if the other is up).
- [ ] **1.3** `run-matrix.sh`: for backend in (goapi, apiapp): `backend_up` → for subset in 31: fresh install (`adb uninstall digital.vasic.lava.client.dev || true; adb install -r <apk>`) → start recording → run `Challenge70` with `qa_backend/qa_providers/qa_query` for both `1080p` and `mp3` → stop recording → collect attestation → `backend_down`. Honor IPTorrents policy (Jackett on goapi; attempt+log on apiapp).
- [ ] **1.4** `aggregate-evidence.sh`: roll per-iteration `real-device-verification.json` rows into `summary.md` + per-iteration pass/fail table.
- [ ] **1.5** Falsifiability rehearsal for `Challenge70` (break onboarding provider-select OR the download path; confirm RED with a clear message; revert; GREEN). Record in the test KDoc + commit body Bluff-Audit stamp.

# Phase 2 — Go API backend full matrix (31 × 2 queries)

- [ ]

- [ ] **2.1** Bring Jackett+FlareSolverr sidecar up for IPTorrents (`docker-compose.jackett.yml`, profiles jackett+cloudflare); validate `/caps`.
- [ ] **2.2** Run `run-matrix.sh goapi` (62 iterations). Record real evidence per iteration.
- [ ] **2.3** Triage failures → systematic-debugging → fix root cause → add/strengthen the covering Challenge/bank (reproduce-first) → re-run the failed iterations until green. Log each fix in `docs/issues/fixed/BUGFIXES.md` (§6.T.4).
- [ ] **2.4** Commit harness + curated evidence; push (GitHub+GitLab).

# Phase 3 — Android `:api-app` backend full matrix (31 × 2 queries)

- [ ] **3.1** Build `:api-app` debug APK (`./gradlew :api-app:assembleDebug`).
- [ ] **3.2** Run `run-matrix.sh apiapp` (62 iterations); IPTorrents attempts logged as real findings.
- [ ] **3.3** Triage/fix/reproduce-first/re-run as Phase 2.3. Distinguish genuine `:api-app` limitations (e.g. Cloudflare) from real bugs.
- [ ] **3.4** Commit + push.

# Phase 4 — Vision analysis + endless fix loop

- [ ] **4.1** `vision-analyze.sh`: HelixQA OCR over every recording; emit `vision-analysis.md` per iteration + a cycle-level rollup of defects (white-on-white, stuck spinners, wrong screen, missing download affordance — the §6.AB non-crashing classes).
- [ ] **4.2** For each vision-found defect: reproduce-first device Challenge (RED on current build) → fix → GREEN → BUGFIXES.md entry.
- [ ] **4.3** Loop driver (`ScheduleWakeup`): re-run failed iterations + re-analyze until N consecutive clean passes, or operator

interrupt. Each loop bumps the curated evidence + CONTINUATION.

# Phase 5 — Gate + version bump + distribute (dev + prod)

- [ ] **5.1** Run the FULL §6.AE multi-AVD matrix on the covering Challenges (API 28/30/34/latest × phone+tablet) → `real-device-verification.json` with all gate rows pass + `runner=containers-submodule`.
- [ ] **5.2** §6.Y version bumps: `app/build.gradle.kts` (client), `api-app/build.gradle.kts`, `lava-api-go/internal/version/version.go`; CHANGELOG entries.
- [ ] **5.3** §6.AK cycle-coverage: author `cycle-coverage-map-<ver>.yaml` (every CHANGELOG claim → covering executed Challenge) + `<ver>-test-evidence.json` (same SHA, executed PASS rows); `check-cycle-coverage.sh --strict` exits 0.
- [ ] **5.4** §6.Z evidence file incl. `combined-distribute-authorization` (operator pre-authorized) + cold-start (C00) PASS for each APK.
- [ ] **5.5** Two-stage distribute: `firebase-distribute.sh --debug-only` (client + api-app dev) → then prod, per operator's auto-both authorization. Verify §6.P monotonic versionCode + CHANGELOG.
- [ ] **5.6** Commit + push everything (submodules + main → GitHub + GitLab). Final report to operator with distribute IDs + evidence index.

---

# Anti-bluff / constitutional gate checklist (run before each commit)

- `scripts/check-constitution.sh` (forbidden cmds, creds, no-hardcode, §6.W mirrors)
- `scripts/check-challenge-coverage.sh` + `scripts/check-challenge-discrimination.sh`
- `./scripts/ci.sh --changed-only` (pre-push subset)
- No raw media staged (`git status` shows only curated `.md`/`.json` under `autonomous-qa/`)

# Execution mode

Subagent-driven for parallel BUILD (Challenge author, bank generator, orchestrator scripts, vision tooling, gate wiring) + parallel vision ANALYSIS of recordings. The emulator/backend RUNTIME loop runs inline/sequentially (one emulator, one backend). Phase 0 runs first, inline, as the keystone proof.